Project:

Hospital Management Database Analysis

Objective:

Exploring and analysing a hospital's management database/records using MySQL

PART 1: DATASET OVERVIEW

The project involves 4 different tables of 4 different datasets within the database as described below. Each dataset is independent of each other, despite containing the same values for full_name columns:

1. Staff table:
    a. Columns:
        i. Staff_id (INT): unique identifiers of each staff within the hospital
        ii. full_name (VARCHAR): first and last name of each staff
        iii. staff_role (VARCHAR): 4 different staff roles - doctor, nursing assistant and nurse
        iv. service (VARCHAR): 4 different departments within the hospital - ICU, surgery, emergency and general medicine
    b. Number of rows: 110
2. Schedule table:
    a. Columns:
        i. week_num (INT): the number of the week
        ii. Staff_id (VARCHAR): unique identifiers of each staff within the hospital
        iii. full_name (VARCHAR): first and last name of each staff
        iv. staff_role (VARCHAR): 4 different staff roles - doctor, nursing assistant and nurse
        v. service (VARCHAR): 4 different departments within the hospital - ICU, surgery, emergency and general medicine
        vi. Presence (BOOLEAN): marks the presence of each staff using boolean values
    b. Number of rows: 6552
3. Service table:
    a. Columns:
        i. week_num (INT): the number of the week
        ii. month_num (INT): the number of the month
        iii. service (VARCHAR): 4 different departments within the hospital - ICU, surgery, emergency and general medicine
        iv. available_beds (INT): number of beds available within each department per week
        v. request (INT): number of patients requesting admission within each department per week
        vi. admitted (INT): number of patient admitted to the hospital within each department per week

vii. refused (INT): number of patient refused admission within each department per week
viii. patient_satisfaction (INT): average patient satisfaction of each department per week
ix. staff_morale (INT): average staff morale of each department per week
x. event_occured (VARCHAR): types of event occurring within each department per week - none, flu, donation and strike

b. Number of rows: 208

4. Patients table:
   a. Columns:
      i. patient_id (VARCHAR): unique identifiers of patients admitted to the hospital
      ii. full_name (VARCHAR): first and last name of patients
      iii. age: patient's age
      iv. arrival (DATE): date of patient's admission to the hospital
      v. departure (DATE): date of patient's discharge from the hospital
      vi. service (VARCHAR): department patient is admitted to - ICU, general medicine, surgery and emergency
      vii. satisfaction (INT): patient's satisfaction of the service provided by the hospital
   b. Number of rows: 1000

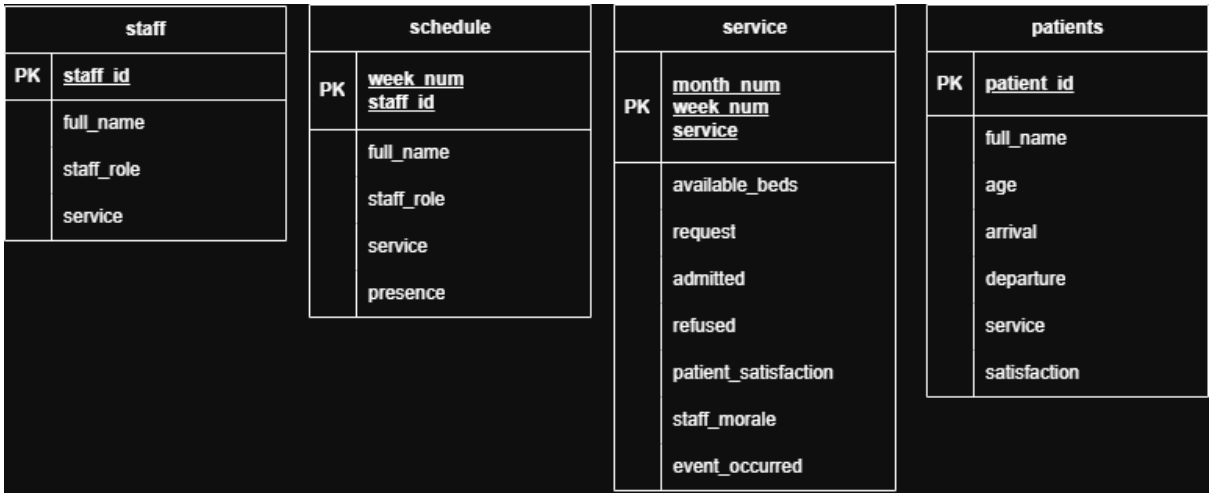

Figure 1: Entity relationship diagram

PART 2: GOALS, QUERIES AND FINDINGS

Table: Staff

1. **How many staff are assigned to each role?**

```
SELECT
        staff_role,
        COUNT(*) AS value_count
FROM staff
GROUP BY staff_role;
```

The query counts the number of rows of the columns and groups them by the staff's role. This returns the following results:

staff_role	value_count
doctor	18
nursing_assistant	23
nurse	69

We can see that the role with the highest assignment is nurse and the least is doctor. This puts the doctor-nurse ratio at 6:23, which translates to "for every one doctor, three to four nurses are assigned together". Although there isn't an ideal ratio for doctor-to-nurse, this ratio ensures that a bottleneck is not created that leads to task failure, whether due to high medical orders from doctors cannot be executed at the same pace or high clinical decisions delay due to doctor's unavailability to sign off a case.

For nurse-to-nursing-assistant on the other hand, we have a 1:3 ratio, translating to "each nurse is assigned to three nursing assistants". This is ideal in ensuring that each nurse is able to supervise their nursing assistants properly without missing proper care for patients and nursing assistants are not burnt out from mundane tasks due to overstaffing.

2. **How many staff are assigned to each department?**

```
SELECT
        service,
   COUNT(*) AS value_count
FROM staff
GROUP BY service;
```

The query counts each row of each column and groups them by service (departments). Following is the result:

service	value_count
ICU	32
surgery	22
emergency	29
general_medicine	27

We can see that the department with the highest staffing is ICU with 32 staff while the lowest is general medicine with 27 staff. Take note that more staff might need to be hired in the future to ensure proper staff-to-patient ratio to provide quality medical service and lower the mortality rate.

Table: schedule

1. **How many presence and absence does each staff member have?**

```
SELECT
        full_name, staff_id, staff_role, service,
        COUNT(CASE WHEN presence=1 THEN 1 END) AS true_count,
        COUNT(CASE WHEN presence=0 THEN 1 END) AS false_count
FROM schedule
GROUP BY full_name, staff_id, staff_role, service
ORDER BY true_count DESC;
```

The query counts the boolean values for presence of each staff member, where if presence = 1, it indicates that the staff is present and totals the presence up as true_count, while if presence = 0, it indicates the staff is absent and totals the absence up as false_count. The result is ordered by highest number of presence to lowest for each staff member. Following is the result for highest presence and lowest:

full_name	staff_id	staff_role	service	true_count	false_count
Monica Herrera	STF-4e69ee46	nurse	emergency	34	18
Holly Wood	STF-b0bc8d3f	nurse	emergency	34	18
Stephanie Ross	STF-b13eeb99	nurse	emergency	34	18
Donald Wright	STF-e2bbe735	nursing_assistant	surgery	34	18
Daniel Baker	STF-e72deb85	nursing_assistant	general_medicine	34	18
Angela Lopez	STF-0f188974	nurse	general_medicine	28	24
Amber Kidd	STF-5157d809	doctor	general_medicine	28	24
Noah Rhodes	STF-5e560b99	doctor	emergency	28	24

There are five staff members with the highest presence count, all have the roles of either nurse or nursing assistant. Their presence amounts to 65.38% of the schedule. Meanwhile, there are three staff members with the lowest presence count of only 53.85%, just a little bit higher than half of the scheduled time. Further investigation on staff's absences needed to be done to ensure proper actions are taken if absences are not justified. Especially because two of the least present staff members are doctors and one of them is an emergency department's doctor, where high traffic and urgent care for patients are expected. Unjustified absences can skew the proper staff-member-to-patient ratio, which can lead to improper medical care and higher mortality rate.

2. **How many staff are scheduled to work each week?**

```
WITH ranked AS (
        SELECT
                week_num, service,
```

```
                COUNT(staff_id) OVER (PARTITION BY week_num, service) AS
number_of_staff,
                ROW_NUMBER() OVER (PARTITION BY week_num, service) AS rn
        FROM schedule
)
SELECT
        week_num, service, number_of_staff
FROM ranked
WHERE rn = 1;
```

The query uses CTE to calculate the number of staff working each week by departments and allows the use of window function ROW_NUMBER() to show the first row of number_of_staff values only. This allows easier result readability because the result will not return the same value multiple times only because there are different staff_id. The result is shown as follows:

week_num	service	number_of_staff
1	emergency	39
1	general_medicine	28
1	ICU	34
1	surgery	25

The screenshot provided is only for the first week because upon further investigation of the result, the number of staff scheduled to each department is the same each week, with the highest being in the emergency department and lowest being in the surgery department.

3. **How many staff members are present each week by departments**

```
WITH ranked AS (
        SELECT
                *,
      ROW_NUMBER() OVER (PARTITION BY week_num, service ORDER BY
week_num) AS rn,
                COUNT(CASE WHEN presence=1 THEN 1 END) OVER (PARTITION
BY week_num, service) AS true_count,
                COUNT(CASE WHEN presence=0 THEN 1 END) OVER (PARTITION
BY week_num, service) AS false_count
      FROM schedule
)
SELECT
        week_num, service, true_count, false_count
FROM ranked
WHERE rn = 1 AND true_count != 0
ORDER BY week_num;
```

The query uses CTE to calculate the number of presences and absences within each department by week.

This query allows management to see trends of absences within each department to facilitate hiring and scheduling processes. For example, in week 5, 6 staff members are absent from the ICU department and 5 are absent from the general medicine department. Absences should be investigated, and if needed, more staff members can be hired for departments with frequent absences to ensure good staff-member-to-patient ratio.

Note that 3 weeks, the presence for each department is 0. This could be an error in the database, hence further investigation is needed to find out why this occurs. The view is removed from the result grid using the WHERE clause.

4. **Which event has the highest occurrence each week?**

```
WITH ranked AS (
        SELECT
                week_num, month_num, event_occured,
    COUNT(event_occured) AS occurrence,
                DENSE_RANK() OVER (
                        PARTITION BY week_num
                        ORDER BY COUNT(event_occured) DESC) as ranking
        FROM service
  GROUP BY week_num, month_num, event_occured)
SELECT
        week_num, month_num, event_occured, occurrence
FROM ranked
WHERE
        ranking = 1
GROUP BY week_num, month_num, event_occured, occurrence
ORDER BY week_num ASC, occurrence DESC;
```

The query uses CTE to count the number of events occurring each week and orders them using the DENSE_RANK() function to find the event with highest occurrence each week.

It helps management see trends in events occurring to better prepare staff and facilitate scheduling processes. However, the result shows that for most weeks, most departments experience no occurring events.

5. **How many patients are admitted each week?**

```
SELECT
        week_num, month_num,
  SUM(admitted) AS number_admitted
FROM service
GROUP BY week_num, month_num;
```

The query calculates the total admission into the hospital each week.

The result can facilitate management in hiring processes and asset acquisition. For example, if admission is high, management should invest in hiring more staff members to provide quality medical care as well as invest in more beds to ensure that

all patients with high needs can be admitted into the hospital instead of turned away due to full occupancy.

6. **What is the average number of admission each month?**

```
SELECT
        month_num,
   AVG(admitted) AS average_admitted
FROM service
GROUP BY month_num
ORDER BY average_admitted DESC;
```

The query calculates the average number of admitted patients each month and sorts it in descending order.

It serves the same purpose as Question 5.

7. **Does patient refusal of admission affect satisfaction?**

```
WITH base AS(
        SELECT
                *,
                (refused * 100) / request AS refused_percentage
        FROM service
),
classification AS(
        SELECT
                *,
                CASE
                        WHEN patient_satisfaction >= 50 AND refused_percentage
<= 50
                                THEN 'Affected'
                        WHEN patient_satisfaction < 50 AND refused_percentage >
50
                                THEN 'Affected'
                        ELSE 'Not affected'
                END AS Possibility_of_Effect
        FROM base
),
summary AS (
        SELECT
                SUM(Possibility_of_Effect = 'Affected') AS Affected,
                SUM(Possibility_of_Effect = 'Not affected') AS Not_affected
        FROM classification
)
SELECT
        month_num, week_num, service, refused_percentage,
   patient_satisfaction, Possibility_of_Effect
```

```
FROM classification
UNION ALL
SELECT
        'Totals', '', '', '', Affected, Not_affected
FROM summary;
```

The query uses CTE to calculate the percentage of refusal, classifies it as whether patient satisfaction is affected by refusal percentage (where if satisfaction <= 50 and refusal >= 50 or satisfaction >50 and refusal<50, it indicates that both are moving in opposite direction, indicating inverse relationship between two variables) and summarises the classification into number of cases where relationship possibly occurs. The following is the total of cases possibly affected by the relationship and possibly not affected by the relationship:

Totals	136	72

We can see that the total cases affected by the relationship is higher than total cases not affected by the relationship. This indicates a possible inverse relationship between the two variables. However, a conclusion cannot solely be made based on these results since there are limitations on the cases, where even a tiny change in the parameters can skew the result to either 'affected' or 'not affected' relationship, hence further investigation needs to be made using data analysis methods such as Pearson Correlation.

Even so, this result can still help management in asset acquisition decisions. Since more cases seem to be affected by the inverse relationship, it is possible that patients who requested to be admitted but are refused due to bed occupancy may experience lower satisfaction. Therefore, the management may consider investing in more beds to admit patients, consequently more medical staff to make up for the staff-member-to-patient ratio.

8. **Which week/month has the highest/lowest staff morale and does an event occurring affects the staff's morale?**

```
WITH base AS (
        SELECT
                *,
                CASE
                        WHEN event_occured != 'none' THEN 1
                        ELSE 0
                END AS 'Event'
        FROM service
),
ranked AS (
        SELECT
                *,
```

```
        DENSE_RANK() OVER (PARTITION BY week_num ORDER BY month_num,
staff_morale DESC) AS rn
    FROM base
),
summary AS (
        SELECT
                *,
                CASE
                        WHEN Event = 1 and staff_morale < 50 THEN 'Affected'
        WHEN Event = 0 and staff_morale >= 50 THEN 'Affected'
        ELSE 'Not affected'
                END AS effects
        FROM ranked
),
total_summary AS (
        SELECT
                SUM(effects = 'Affected') AS total_affected,
      SUM(effects = 'Not affected') AS total_not_affected
        FROM summary
)
SELECT
        week_num, month_num, service, Event, staff_morale
FROM summary
WHERE rn = 1
UNION ALL
SELECT
        'total', '', '', total_not_affected, total_affected
FROM total_summary
ORDER BY staff_morale DESC;
```

The query uses CTE to turn event occurrence into a boolean variable, use DENSE_RANK() to sort the results into highest and lowest staff morale of the week without excluding tie staff morales between departments, classifies whether staff morale is affected by an event or not similar to Question 7 and finds the total of cases where inverse relationship occurs. Result is as follows:

week_num	month_num	service	Event	staff_morale
total			38	170
36	9	surgery	0	99
29	8	emergency	0	99
37	10	surgery	0	66

The highest staff morale occurs at week 36 and 29 for the departments surgery and emergency respectively, where the average morales are 99 and no events occurred

on said week. The lowest morale average occurs at week 37 for the surgery department where no events occurred yet the average staff morale is 66. We can also see that the total number of cases affected by an inverse relationship between the two variables is 170, which is higher than the total cases not affected. Similar limitations from Question 7 also apply here.

However, the insight can help deduce that it is more likely that average staff morale will be lower when an event is occurring. Further investigation needs to be done on why this is occurring. A few suggestions that may cause this could be staff member's burnout or fatigue during the event. In this case, management may opt to schedule more staff members to the department where an event is expected to occur or is currently occurring.

9. Does event occurrence affect patient admission?

```
WITH base AS (
        SELECT
                week_num, month_num, event_occured, available_beds, admitted,
service
        FROM service
),
averaging AS (
SELECT
        week_num, month_num, event_occured, available_beds, admitted, service,
   AVG(admitted) OVER (PARTITION BY month_num, service) AS average
FROM base
),
class AS (
        SELECT
      week_num, month_num, event_occured, available_beds, admitted, average,
service,
                CASE
                        WHEN event_occured != 'none' AND admitted > average
THEN 'Affected'
        ELSE 'Not affected'
                END AS Effect
        FROM averaging
   GROUP BY week_num, month_num, event_occured, available_beds, admitted,
average, service
)
SELECT
        SUM(Effect = 'Affected') AS total_affected, SUM(Effect = 'Not affected') AS
total_not_affected
FROM class;
```

The query uses CTE to find the average number of admissions each month, classifies whether admission is affected by event occurrence or not (where if an event occurs

and the admission of the day is higher than the month's average or vice versa, it is considered that an inverse relationship occurs) and totalling the number of cases where an inverse relationship occurs or not. The result of the query is as follows:

total_affected	total_not_affected
18	190

We can see here that in most cases, the inverse relationship does not occur, therefore an event occurring does not increase nor decrease a patient's need to be admitted. However, note that model limitation from Question 7 still applies here.

Table: Patients

1. **How many patients are admitted each month and what is the average admission per week?**

```
WITH base AS (
        SELECT
                patient_id, full_name, age, arrival, departure, service, satisfaction,
    EXTRACT(YEAR FROM arrival) AS arrival_year,
    EXTRACT(MONTH FROM arrival) AS arrival_month,
    WEEK(arrival,4) AS arrival_week
        FROM patients
  ORDER BY arrival_year, arrival_month, arrival_week
),
ranked AS (
        SELECT
                *,
                ROW_NUMBER() OVER (PARTITION BY arrival_week ORDER BY
arrival_week) AS rn
        FROM base
),
calculate AS (
        SELECT
                *,
                COUNT(patient_id) OVER (PARTITION BY arrival_month) AS
patient_arrived_month,
    COUNT(patient_id) OVER (PARTITION BY arrival_week) AS
patient_arrived_week
        FROM ranked
)
SELECT
        arrival_month, arrival_week, patient_arrived_month, patient_arrived_week,
  AVG(patient_arrived_week) OVER (PARTITION BY arrival_month) AS
average_patient_week
FROM calculate
```

```
WHERE rn = 1;
```

The query uses CTE to separate month, year and week from the date column and calculates the total of patients admitted each month and average patient admitted each week.

This query helps management to see a trend in patient admission and decide on scheduling and hiring processes. The month with the highest total patient is month 9 at 94 patients and the lowest is in month 12 at 74 patients. However, the highest weekly average admission is in month 10 at 22.5 admissions per week and the lowest is in month 1 at 17.4 admissions per week

2. **Youngest and oldest patient of each month**

```
WITH base AS (
        SELECT
                patient_id, full_name, age, service,
    EXTRACT(YEAR FROM arrival) AS arrival_year,
    EXTRACT(MONTH FROM arrival) AS arrival_month
        FROM patients
  ORDER BY arrival_year, arrival_month
),
ranked AS (
        SELECT
                arrival_month, patient_id, full_name, service,
                MIN(age) OVER (PARTITION BY arrival_month, service) AS
youngest_patient,
                MAX(age) OVER (PARTITION BY arrival_month, service) AS
oldest_patient,
    ROW_NUMBER() OVER (PARTITION BY arrival_month, service) AS rn
        FROM base
        ORDER BY arrival_month, service
)
SELECT
        arrival_month, patient_id, full_name, service, youngest_patient,
oldest_patient
FROM ranked
WHERE rn = 1;
```

The query uses CTE to extract month and year from the date column, and finds the youngest and oldest patient age for each month.

3. **What is the average admission duration of each month?**

```
WITH base AS (
        SELECT
                patient_id, full_name, age, arrival, departure, service,
    DATEDIFF(departure, arrival) AS duration,
    EXTRACT(MONTH FROM arrival) AS month_num,
```

```
        WEEK(arrival,4) AS week_num
            FROM patients
),
ranked AS (
            SELECT
                    month_num, week_num, service,
        AVG(duration) OVER (PARTITION BY month_num, week_num, service) AS
average_stay_duration,
        ROW_NUMBER() OVER (PARTITION BY month_num, week_num, service) AS
rn
            FROM base
    ORDER BY month_num, week_num, service
)
SELECT
            month_num, week_num, service, average_stay_duration
FROM ranked
WHERE rn = 1
ORDER BY month_num, week_num, service;
```

The query gives insight on the average admission duration to help management in asset acquisition processes (since longer admission = longer bed occupancy, hence more bed might be needed to admit urgent care patients) and scheduling processes (to ensure each patient is taken care of properly).

4. **Does admission duration affect patient satisfaction?**

```
WITH base AS (
            SELECT
                    service, satisfaction,
        EXTRACT(MONTH FROM arrival) AS month_num,
        WEEK(arrival, 4) AS week_num,
        DATEDIFF(departure, arrival) as duration
            FROM patients
),
calculate AS (
            SELECT
                    month_num, week_num, service, duration, satisfaction,
        AVG(duration) OVER (PARTITION BY month_num, week_num, service) AS
average_duration,
        AVG(satisfaction) OVER (PARTITION BY month_num, week_num, service) AS
average_satisfaction
            FROM base
    ORDER BY month_num, week_num, service
),
classify AS (
            SELECT
```

```
                month_num, week_num, service, satisfaction,
        CASE
                        WHEN duration >= average_duration AND satisfaction <=
average_satisfaction THEN 'Affected'
        WHEN duration < average_duration AND satisfaction > average_satisfaction
THEN 'Affected'
        ELSE 'Not affected'
                END AS Possibility_of_effect
        FROM calculate
)
SELECT
        SUM(Possibility_of_effect = 'Affected') AS total_affected,
    SUM(Possibility_of_effect = 'Not affected') AS total_not_affected
FROM classify;
```

The query extracts the month from the date column, divides cases into weeks, calculates the average satisfaction and average admission duration and classifies each case into whether satisfaction and admission have an inverse relationship or not. Result is as follows:

total_affected	total_not_affected
491	509

We can see that for most cases, an inverse relationship does not apply. However, model limitations from Question 7 of table schedule also apply here.

PART 3: MODEL LIMITATIONS AND CONCLUSION

As already discussed several times in the previous section, conclusive deduction cannot be made whether a variable has a relationship with another variable due to model limitations, where the thresholds set for each variable may skew the results into either classes even at the tiniest change. Hence, further investigation on variable relationships have to be done on a separate project or software such as Python to discover hidden relationships using methods such as Pearson Correlation Matrix.

The datasets provided also only span for a year, thus predictive analysis cannot be done properly. It is preferable for the dataset to contain a few years worth of data in order to properly see trends such as which month will always have the lowest staff morale or when the patient's admission duration is expected to be longest. These predictive analysis will help the hospital and management make better decisions at hiring, scheduling and asset acquisition.

Further investigation is also needed to discover why the given dataset for "schedule" is missing presence data every 3 weeks.

For future projects, it is highly recommended to involve other data analysis softwares and predictive analysis softwares in order to help make better decisions for management. It is also important to note that two variables can be correlated not only through inverse relation but also linear relation.